

SENA

Servicio Nacional de Aprendizaje

TALLER N° 1

ENCAPSULAMIENTO

Programación Orientada a Objetos en Java

Pilar de la POO: Encapsulamiento

Lenguaje: Java

Nivel: Básico

Ejercicios: 5 ejercicios prácticos + 1 reto integrador

Metodología: Constructivista (aprender-haciendo)

Modalidad: Clases y métodos main en archivos separados

ELABORADO POR

Carlos Arturo Barrientos Suares

*Ingeniero de Sistemas · Especialista en Desarrollo de Software · Máster en Ingeniería de Software y Sistemas
Informáticos*

Marco conceptual del taller

¿Qué es el encapsulamiento?

Imagina un cajero automático. Tú puedes **consultar tu saldo**, **retirar** o **consignar** dinero, pero no puedes abrir la máquina y modificar los billetes con tus manos. La máquina *oculta* su funcionamiento interno y solo te permite hacer operaciones controladas. Eso, en Programación Orientada a Objetos, se llama **encapsulamiento**: proteger los datos internos de un objeto y permitir modificarlos únicamente a través de operaciones controladas.

¿Cómo se aplica en Java?

En Java, el encapsulamiento se logra con dos herramientas fundamentales: usar el modificador **private** en los atributos (para que nadie los toque directamente desde fuera) y crear métodos públicos llamados **getters** (para consultar) y **setters** (para modificar con validaciones). Es como decir: *"si quieres cambiar mi saldo, tienes que pedírmelo, y yo decido si te dejo o no"*.

¿Por qué es tan importante?

1. **Seguridad de los datos**: nadie puede asignar valores absurdos (una edad negativa, un stock de -100 unidades).
2. **Validación centralizada**: las reglas del negocio viven dentro de la clase, no repartidas por todo el programa.
3. **Mantenimiento**: si cambias cómo guardas un dato por dentro, no afectas al código que usa la clase.
4. **Reutilización**: una clase bien encapsulada se puede usar en otros proyectos sin miedo.
5. **Menos errores**: el compilador y tus métodos protegen los datos automáticamente.

METODOLOGÍA DEL TALLER

No copies las soluciones sin pensar. Cada ejercicio empieza con una **situación problema**, seguida de **preguntas para activar tu mente**. Intenta responderlas antes de mirar el código. Al final tendrás un **reto de construcción** donde tú aportas piezas del código. Recuerda: *los conocimientos que construyes con tus manos son los que realmente aprendes*.

Ejercicio 1: Cuenta Bancaria de Bancolombia

SITUACIÓN PROBLEMA

Trabajas como desarrollador Java en un banco colombiano. El banco necesita representar digitalmente las cuentas de sus clientes. Cada cuenta tiene un **número de cuenta**, un **titular** y un **saldo**. El equipo de seguridad informática ha establecido tres reglas **obligatorias**:

- El saldo **NUNCA** debe poder modificarse directamente desde fuera de la clase.
- Solo se puede depositar dinero en cantidades **positivas**.
- Solo se puede retirar si **hay saldo suficiente**.

Tu tarea: diseñar la clase **CuentaBancaria** que cumpla estas reglas.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de mirar la solución, piensa y responde en tu cuaderno:

1. ¿Qué pasaría si el atributo *saldo* fuera público? ¿Un empleado deshonesto podría cambiarlo?
2. ¿Dónde debería estar la lógica que valida si hay saldo suficiente para retirar: dentro de la clase o en el código que la usa?
3. Si el banco cambia mañana la regla y permite descubierto de \$50.000, ¿cuántos archivos tendrías que modificar si el saldo es público?

Solución guiada

Vamos a construir la clase paso a paso. Fíjate en tres decisiones importantes: (a) todos los atributos van con **private**, (b) los getters devuelven información pero no permiten modificar, y (c) los métodos *depositar* y *retirar* tienen validaciones antes de tocar el saldo.

Archivo: CuentaBancaria.java

```
public class CuentaBancaria {
    // Atributos privados - encapsulados
    private String numeroCuenta;
    private String titular;
    private double saldo;

    public CuentaBancaria(String numeroCuenta, String titular) {
        this.numeroCuenta = numeroCuenta;
        this.titular = titular;
        this.saldo = 0.0;
    }

    // Getters - solo lectura desde el exterior
    public String getNumeroCuenta() { return numeroCuenta; }
    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }

    public void depositar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
            System.out.println("Deposito exitoso de $" + cantidad);
        } else {
            System.out.println("Error: la cantidad debe ser positiva.");
        }
    }

    public void retirar(double cantidad) {
        if (cantidad <= 0) {
            System.out.println("Error: la cantidad debe ser positiva.");
        } else if (cantidad > saldo) {
            System.out.println("Error: saldo insuficiente.");
        } else {
            saldo -= cantidad;
            System.out.println("Retiro exitoso de $" + cantidad);
        }
    }
}
```

Archivo: MainCuenta.java

```
public class MainCuenta {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("001-2345", "Carlos Perez");

        System.out.println("Titular: " + cuenta.getTitular());
        System.out.println("Saldo inicial: $" + cuenta.getSaldo());

        cuenta.depositar(500000);
        cuenta.retirar(150000);
        cuenta.retirar(1000000); // Debe fallar por saldo insuficiente

        System.out.println("Saldo final: $" + cuenta.getSaldo());
    }
}
```

BENEFICIO APLICADO

El atributo *saldo* está protegido. Nadie puede escribir *cuenta.saldo = -999* desde fuera porque el compilador Java lo prohibirá. Toda modificación pasa por métodos con validaciones. Esto es exactamente lo que hacen los bancos reales para proteger tu dinero.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Amplía la clase agregando un método **transferir(CuentaBancaria destino, double cantidad)** que retire dinero de esta cuenta y lo deposite en la cuenta destino. Piensa: ¿qué validaciones debe hacer? ¿Qué pasa si la cantidad es negativa o si no hay saldo suficiente? Prueba tu método transfiriendo dinero entre dos cuentas en el *main*.

Ejercicio 2: Aprendiz del SENA

SITUACIÓN PROBLEMA

El coordinador académico de tu centro SENA te pide un programa para registrar aprendices. Cada aprendiz tiene **número de documento**, **nombre**, **edad** y una **nota final**. El sistema debe garantizar que:

- La edad esté entre **0 y 100** años (imposible tener 200 años o -5).
- La nota esté entre **0.0 y 5.0** (rango vigente en el SENA).
- Se pueda saber automáticamente si el aprendiz **aprobó** (nota mayor o igual a 3.5).

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona antes de codificar:

1. Si un instructor por error escribe una nota de 7.5, ¿tu programa debe aceptarla o rechazarla?
2. ¿Es mejor mostrar un mensaje de error o cambiar silenciosamente el valor?
3. ¿Por qué los setters son tan importantes en este ejercicio?

Solución guiada

Archivo: Aprendiz.java

```
public class Aprendiz {
    private String documento;
    private String nombre;
    private int edad;
    private double notaFinal;

    public Aprendiz(String documento, String nombre) {
        this.documento = documento;
        this.nombre = nombre;
        this.edad = 0;
        this.notaFinal = 0.0;
    }

    public String getDocumento() { return documento; }
    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
    public double getNotaFinal() { return notaFinal; }

    public void setEdad(int edad) {
        if (edad >= 0 && edad <= 100) {
            this.edad = edad;
        } else {
            System.out.println("Edad invalida. Debe estar entre 0 y 100.");
        }
    }

    public void setNotaFinal(double nota) {
        if (nota >= 0.0 && nota <= 5.0) {
            this.notaFinal = nota;
        } else {
            System.out.println("Nota invalida. Debe estar entre 0.0 y 5.0.");
        }
    }

    public boolean aprobo() {
        return notaFinal >= 3.5;
    }
}
```

Archivo: MainAprendiz.java

```
public class MainAprendiz {  
    public static void main(String[] args) {  
        Aprendiz a = new Aprendiz("1090123456", "Laura Gomez");  
  
        a.setEdad(19);  
        a.setNotaFinal(4.2);  
        a.setNotaFinal(7.0); // Invalido - sera rechazado por la clase  
  
        System.out.println("Aprendiz: " + a.getNombre());  
        System.out.println("Edad: " + a.getEdad());  
        System.out.println("Nota: " + a.getNotaFinal());  
        System.out.println("Aprobo? " + (a.aprobo() ? "Si" : "No"));  
    }  
}
```

BENEFICIO APLICADO

Los setters actúan como **guardianes** de los datos. Aunque el código externo intente asignar valores inválidos, la clase los rechaza. Un objeto *Aprendiz* **siempre** estará en un estado válido. En sistemas reales, esto evita errores gravísimos como reportar que un aprendiz tiene 200 años.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade a la clase **Aprendiz** un atributo privado **correoInstitucional** con su getter y setter. El setter debe validar que el correo **contenga el símbolo @** antes de asignarlo. Pista: usa `correo.contains("@")`. Pruébalo en el *main* intentando asignar un correo inválido y uno válido.

Ejercicio 3: Inventario de una Tienda

SITUACIÓN PROBLEMA

Una tienda de barrio en Cúcuta te pide un sistema para gestionar sus productos. Cada producto tiene **código**, **nombre**, **precio** y **cantidad en stock**. El dueño quiere estar tranquilo de que:

- El precio **nunca** sea negativo.
- El stock **nunca** quede por debajo de cero (no se puede vender lo que no hay).
- Cuando alguien vende, el stock **se descuenta automáticamente**.
- Cuando llegue mercancía nueva, se pueda **reabastecer**.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿Qué pasaría si en el *main* alguien escribe *producto.stock = -50* sin encapsulamiento?
2. ¿La validación del stock debería estar en el método *vender*, o repetirse en cada parte del código que venda?
3. ¿Por qué es útil que el constructor use los setters para inicializar el precio y el stock?

Solución guiada

Archivo: Producto.java

```
public class Producto {
    private String codigo;
    private String nombre;
    private double precio;
    private int stock;

    public Producto(String codigo, String nombre, double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        setPrecio(precio); // Usamos el setter para validar
        setStock(stock);
    }

    public String getCodigo() { return codigo; }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
    public int getStock() { return stock; }

    public void setPrecio(double precio) {
        if (precio >= 0) this.precio = precio;
        else System.out.println("Precio invalido.");
    }

    public void setStock(int stock) {
        if (stock >= 0) this.stock = stock;
        else System.out.println("Stock invalido.");
    }

    public void vender(int unidades) {
        if (unidades <= 0) {
            System.out.println("Cantidad invalida.");
        } else if (unidades > stock) {
            System.out.println("Sin stock suficiente. Disponible: " + stock);
        } else {
            stock -= unidades;
            System.out.println("Venta realizada: " + unidades + " unidades.");
        }
    }

    public void reabastecer(int unidades) {
        if (unidades > 0) {
            stock += unidades;
            System.out.println("Stock actualizado. Nuevo stock: " + stock);
        }
    }
}
```

Archivo: MainProducto.java

```
public class MainProducto {  
    public static void main(String[] args) {  
        Producto p = new Producto("P-001", "Teclado Mecanico", 180000, 10);  
  
        System.out.println("Producto: " + p.getNombre());  
        System.out.println("Precio: $" + p.getPrecio());  
        System.out.println("Stock: " + p.getStock());  
  
        p.vender(3);  
        p.vender(20); // Debe fallar  
        p.reabastecer(5);  
  
        System.out.println("Stock final: " + p.getStock());  
    }  
}
```

BENEFICIO APLICADO

El stock nunca podrá ser negativo. La lógica de negocio (validar cantidades, controlar stock) vive **centralizada** dentro de la clase. Si mañana la tienda abre 3 sucursales más, ninguna de ellas podrá romper las reglas porque los datos están encapsulados.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Agrega dos métodos: **aplicarDescuento(double porcentaje)** que reduzca el precio en el porcentaje indicado (rechaza porcentajes fuera de 0-100), y **hayStock()** que devuelva *true* si el stock es mayor a 0. Úsalos en el main para simular un descuento del 15% y luego verificar si hay stock disponible.

Ejercicio 4: Vehículo de Transporte

SITUACIÓN PROBLEMA

Una empresa de transporte de carga en Norte de Santander te contrata para digitalizar el control de sus vehículos. Cada vehículo tiene **placa**, **marca**, **modelo** y una **velocidad actual**. La empresa quiere que:

- La velocidad esté siempre entre **0 y 200 km/h**.
- Los conductores puedan **acelerar** (subir 10 km/h) y **frenar** (bajar 10 km/h).
- Nunca se supere el límite ni la velocidad baje de cero.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de codificar:

1. ¿Debería la clase *Vehiculo* conocer el valor máximo de velocidad, o debería recibirlo desde fuera cada vez?
2. ¿Qué ventaja tiene declarar `VEL_MAX` como **private static final**?
3. Si mañana el gobierno cambia el límite máximo a 180 km/h, ¿dónde lo cambiarías?

Solución guiada

Archivo: Vehiculo.java

```
public class Vehiculo {
    private String placa;
    private String marca;
    private String modelo;
    private int velocidad;

    private static final int VEL_MAX = 200;
    private static final int VEL_MIN = 0;

    public Vehiculo(String placa, String marca, String modelo) {
        this.placa = placa;
        this.marca = marca;
        this.modelo = modelo;
        this.velocidad = 0;
    }

    public String getPlaca() { return placa; }
    public String getMarca() { return marca; }
    public String getModelo() { return modelo; }
    public int getVelocidad() { return velocidad; }

    public void acelerar() {
        if (velocidad + 10 <= VEL_MAX) {
            velocidad += 10;
            System.out.println("Velocidad: " + velocidad + " km/h");
        } else {
            System.out.println("No puede superar " + VEL_MAX + " km/h.");
        }
    }

    public void frenar() {
        if (velocidad - 10 >= VEL_MIN) {
            velocidad -= 10;
            System.out.println("Velocidad: " + velocidad + " km/h");
        } else {
            velocidad = 0;
            System.out.println("Vehiculo detenido.");
        }
    }
}
```

Archivo: MainVehiculo.java

```
public class MainVehiculo {
    public static void main(String[] args) {
        Vehiculo v = new Vehiculo("ABC123", "Renault", "Logan");

        System.out.println("Vehiculo: " + v.getMarca() + " " + v.getModelo());

        v.acelerar();
        v.acelerar();
        v.acelerar();
        v.frenar();

        System.out.println("Velocidad final: " + v.getVelocidad() + " km/h");
    }
}
```

BENEFICIO APLICADO

El uso de constantes privadas (*VEL_MAX*, *VEL_MIN*) junto con métodos públicos demuestra la potencia del encapsulamiento: si mañana cambia la regla del negocio, se modifica **en un solo lugar** y todo el sistema queda actualizado.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade un atributo privado **kilometrajeTotal** (double) que registre los kilómetros recorridos. Cada vez que el vehículo *acelera*, súmale *velocidad / 100* al kilometraje (simulación simple). Añade un getter y muéstralo al final en el *main*. Extra: crea un método **reset()** que ponga la velocidad en 0.

Ejercicio 5: Autenticación Segura de Usuario

SITUACIÓN PROBLEMA

Estás desarrollando el módulo de login para una aplicación en tu centro SENA. Cada usuario tiene **nombre de usuario** y **contraseña**. Como programador responsable, sabes que:

- La contraseña **NUNCA** debe poder ser leída desde fuera de la clase (ni con getter).
- Solo se puede cambiar si cumple: **mínimo 8 caracteres** y **al menos un número**.
- Se puede **verificar** si una contraseña ingresada coincide con la actual.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. Si existiera un `getPassword()`, ¿qué riesgos habría?
2. ¿Por qué la validación (método `esPasswordValida`) es privada y no pública?
3. ¿Cómo verificamos entonces si un intento de login es correcto, sin exponer la contraseña?

Solución guiada

Archivo: Usuario.java

```
public class Usuario {
    private String nombreUsuario;
    private String password; // Nunca tendra getter publico

    public Usuario(String nombreUsuario, String passwordInicial) {
        this.nombreUsuario = nombreUsuario;
        cambiarPassword(passwordInicial);
    }

    public String getNombreUsuario() {
        return nombreUsuario;
    }

    public void cambiarPassword(String nueva) {
        if (esPasswordValida(nueva)) {
            this.password = nueva;
            System.out.println("Contraseña actualizada correctamente.");
        } else {
            System.out.println("Contraseña invalida: min 8 caracteres " +
                               "y al menos un numero.");
        }
    }

    public boolean verificarPassword(String intento) {
        return this.password != null && this.password.equals(intento);
    }

    private boolean esPasswordValida(String pass) {
        if (pass == null || pass.length() < 8) return false;
        for (char c : pass.toCharArray()) {
            if (Character.isDigit(c)) return true;
        }
        return false;
    }
}
```

Archivo: MainUsuario.java

```
public class MainUsuario {
    public static void main(String[] args) {
        Usuario u = new Usuario("carlos.dev", "clave1234");

        System.out.println("Usuario: " + u.getNombreUsuario());

        u.cambiarPassword("hola"); // Invalida
        System.out.println("Login OK? " + u.verificarPassword("clave1234"));
        System.out.println("Login mal? " + u.verificarPassword("otra"));

        u.cambiarPassword("nuevaClave9");
        System.out.println("Login nuevo? " + u.verificarPassword("nuevaClave9"));
    }
}
```

BENEFICIO APLICADO

Este es el encapsulamiento en su máxima expresión: el atributo *password* **nunca** puede ser leído desde fuera. Solo se puede verificar si una cadena coincide. Esta técnica es real y se usa en sistemas de autenticación profesionales (bancos, redes sociales, apps del gobierno). Sin encapsulamiento, las contraseñas quedarían expuestas.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade un contador privado **intentosFallidos**. Cada vez que *verificarPassword* devuelva *false*, aumenta el contador. Cuando llegue a 3 intentos fallidos, la cuenta debe **bloquearse** (crea un atributo *boolean bloqueado*). Cuando esté bloqueada, *verificarPassword* debe siempre devolver *false*. Pruébalo simulando 3 intentos fallidos en el main.

Reto Integrador: Historia Clínica de una IPS

SITUACIÓN PROBLEMA

La IPS de tu barrio te contrata para el módulo de **historia clínica de pacientes**. Es información confidencial regulada por la **Ley 1581 de 2012** de protección de datos. Debes diseñar la clase **Paciente** con:

- **documento, nombre, edad** (0-120), **tipoSangre** (A+, A-, B+, B-, O+, O-, AB+, AB-).
- Un atributo privado **diagnostico** que **NO tenga getter público**.
- Un método **consultarDiagnostico(String claveMedico)** que solo devuelva el diagnóstico si la clave coincide con "MED-2026".
- Métodos para **actualizarDiagnostico** (solo si la clave es correcta) y **tieneAlergias()**.
- Un atributo **alergias** (arreglo de Strings) con un método para agregarlas.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de resolver, piensa:

1. ¿Qué pasa si un desarrollador crea un `getDiagnostico()` público? ¿Se violaría la ley de protección de datos?
2. ¿Qué otros atributos deberían ser privados con setters validados?
3. ¿Cómo garantizas que la clave del médico no aparezca en logs o mensajes?

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Este reto lo resuelves tú, aplicando **TODO** lo aprendido en los 5 ejercicios:

1. Diseña la clase **Paciente.java** con todos los atributos privados.
2. Los setters validan: edad entre 0-120, tipoSangre solo los 8 valores permitidos.
3. Implementa el sistema de clave del médico para acceder al diagnóstico.
4. Diseña un **MainIPS.java** que:
 - Cree un paciente
 - Intente consultar el diagnóstico con una clave incorrecta
 - Consulte con la clave correcta
 - Agregue 2 alergias
 - Muestre toda la información pública del paciente

Entrega: los dos archivos .java funcionando correctamente. Comparte tu solución con tu instructor para retroalimentación.

Cierre metacognitivo del taller

REFLEXIÓN METACOGNITIVA — ¿QUÉ APRENDÍ?

Antes de terminar, tómate 5 minutos para responder en tu cuaderno:

1. Con tus propias palabras, ¿qué es el encapsulamiento? (evita repetir la definición del libro).
2. ¿En qué ejercicio sentiste que el encapsulamiento se justificaba **más claramente**? ¿Por qué?
3. Menciona un ejemplo **de tu vida cotidiana** donde funcione el mismo principio del encapsulamiento (algo que ocultes o protejas y que solo se pueda acceder de forma controlada).
4. Si tuvieras que explicarle el encapsulamiento a un compañero que faltó a clase, ¿qué analogía usarías?
5. ¿Qué duda te quedó? Anótala para consultarla con tu instructor.

Conclusión del taller

A lo largo de los cinco ejercicios y el reto integrador, no solo has escrito código: has **construido** un modelo mental sólido sobre por qué el encapsulamiento existe y cuándo usarlo. En proyectos profesionales, estas prácticas son **obligatorias**: ningún sistema serio expone sus atributos directamente. Recuerda para siempre: **atributos privados, métodos públicos validados**. En el próximo taller conocerás la **Herencia**, que se apoya en lo que aprendiste hoy.

*SENA - Servicio Nacional de Aprendizaje
Formación Profesional Integral
Programa: Análisis y Desarrollo de Software*